

UNIVERSIDAD AUTÓNOMA DEL ESTADO DE HIDALGO

INSTITUTO DE CIENCIAS BÁSICAS E INGENIERÍA

Licenciatura en Ciencias Computacionales

Autómatas y Compiladores

Práctica: Analizador Léxico

Expresiones Regulares

Alumna:

Pilar Bucio

Semestre:

Sexto semestre

Grupo:

3

Profesor: Eduardo Cornejo Velázquez

1. Introducción

El análisis léxico constituye una de las primeras etapas del proceso de compilación de un programa. Su función principal es analizar el código fuente y dividirlo en unidades léxicas denominadas *tokens*, las cuales posteriormente son utilizadas por las siguientes etapas del compilador.

En esta práctica se aborda la construcción de un analizador léxico básico para código Java mediante el uso de expresiones regulares y las clases `Pattern` y `Matcher` de la biblioteca `java.util.regex`. Se estudia la manera en que un analizador puede identificar diferentes elementos del lenguaje, principalmente palabras reservadas (`KEYWORD`) e identificadores (`IDENTIFIER`).

Además, se analiza la importancia de establecer correctamente el orden de evaluación de los patrones y del uso del límite de palabra `\b`, conocido como *word boundary*. Este permite diferenciar palabras reservadas como `int`, `for` o `return` de identificadores que contienen dichas secuencias, como `integer`, `forLoop` y `returnValue`.

La práctica permite relacionar los conceptos teóricos de expresiones regulares, tokens, lexemas y patrones con una implementación funcional en Java, fortaleciendo la comprensión del funcionamiento de un analizador léxico y de su importancia dentro del proceso de compilación.

2. Objetivos de Aprendizaje

- Distinguir **token, lexema y patrón** en código Java real.
- Escribir expresiones regulares (ER) para cada categoría léxica.
- Implementar un Lexer funcional usando `java.util.regex`.
- Detectar y reportar errores léxicos con número de línea y columna.
- Comprender la relación ER $\leftrightarrow$ AFD $\leftrightarrow$ Tabla de transición.

3. Materiales Necesarios

- JDK 17+ instalado (OpenJDK u Oracle).
- IDE: IntelliJ IDEA Community o VS Code + Java Extension Pack.
- Archivo de prueba `Ejemplo.java`.
- Papel y lápiz para diagramas de autómatas finitos.

4. Marco Teórico

4.1. Los tres conceptos clave

Concepto	Definición	Ejemplo
Token	Par (`tipo, valor`) que representa una unidad léxica con significado. Es el *output* del analizador léxico hacia el parser.	`(KEYWORD, çlass")`
Lexema	Secuencia de caracteres del código fuente que *coincide* con el patrón de un token. Es la cadena cruda leída del archivo.	`çlass"`, `"MiClase"`, `"123"`
Patrón	Regla (expresión regular) que describe el conjunto de cadenas válidas para un token. Define *qué forma puede tener*.	`[a-zA-Z_$]` `[a-zA-Z0-9_$]*`

4.2. Categorías de Tokens en Java

Categoría	Ejemplos de lexemas
KEYWORD	`class if while return int void new public static`
IDENTIFIER	`miVariable _count MiClase $precio`
INTEGER_LIT	`42 0 1_000_000L`
FLOAT_LIT	`3.14 1.5f 2.5e-3d`
STRING_LIT	`"Hola\nMundo"`
CHAR_LIT	`'a' '\n' '\u0041'`
OPERATOR	`+ - * == != && ++ <=`
SEPARATOR	`( ) { } [ ] ; ,`
LINE_COMMENT	`// esto es un comentario`
BLOCK_COMMENT	`/* bloque */`

4.3. Tabla de Patrones (Expresiones Regulares)

Token	Expresión Regular	Descripción
KEYWORD	`\b(class|if|else|while|for|return|int|double|boolean|void|new|public|private|static|final|...)\b`	Palabras reservadas — el `\b` es esencial.
IDENTIFIER	`[a-zA-Z_$][a-zA-Z0-9_$]*`	Letra/_/$ seguido de letras, dígitos o _.
INTEGER_LIT	`0|[1-9][0-9_]*[LI]?`	Decimal: `42`, `1_000_000L`.
FLOAT_LIT	`[0-9][0-9_]*\.[0-9][0-9_]*([eE][+-]?[0-9][0-9_]*)?[fFdD]?`	Con punto decimal, exponente y sufijo opcionales.
STRING_LIT	`"([^"\\]|\\.)*"`	Cadena con secuencias de escape.
CHAR_LIT	`'([^'\\]|\\.)'`	Carácter: `'a'`, `'\n'`.
OPERATOR	`==|!=|<=|>=|&&|||<<|>>|>>>|[/%+\-*=&|^~]`	Compuestos primero, luego simples.
SEPARATOR	`[\(\){}\[\];,]`	Delimitadores y puntuación.
LINE_COMMENT	`//[^\n]*`	Desde `//` hasta fin de línea.
BLOCK_COMMENT	`/\*[\s\S]*?\*/`	Entre `/*` y `*/`.
WHITESPACE	`[\t\r\n]+`	Se consume y se ignora (pero cuenta líneas).

4.4. Reglas de Prioridad (el orden importa)

1. **Comentarios (`//` y `/* */`)** — contienen caracteres que podrían confundir.
2. **Literales de cadena y char** — pueden contener cualquier carácter.
3. **Literales numéricos: FLOAT antes que INT; HEX/BIN/OCT antes que decimal.**
4. **KEYWORD antes que IDENTIFIER** — `while` no debe ser un identificador.
5. **Operadores compuestos antes que simples** — `==` no es `=` + `=`.
6. **Separadores**.
7. **Whitespace** — descartar al final.

Principio de maximal munch: el analizador léxico siempre consume el lexema más largo posible. Así `integer` es un `IDENTIFIER`, no `KEYWORD(ïnt")` + `IDENTIFIER(.eger")`.

5. Desarrollo

5.1. Caso 1 — Básico: Keywords e Identifiers

Nivel: Básico **Tiempo estimado:** 20 min

5.1.1. Objetivo

Dado un fragmento de código Java, identificar y clasificar cada lexema como `KEYWORD` o `IDENTIFIER`, usando expresiones regulares con la API `Pattern/Matcher` de Java.

5.1.2. Patrones

Token	Expresión Regular	Nota
`KEYWORD`	`\b(abstract|assert|boolean|break|...|while|true|false|null)\b`	El `\b` (word boundary) es esencial.
`IDENTIFIER`	`[a-zA-Z_$][a-zA-Z0-9_$]*`	Solo si NO coincidió con `KEYWORD`.

5.1.3. Concepto Clave: Word Boundary `\b`

Sin `\b`, el patrón `int` coincidiría dentro de `integer`, tokenizándolo incorrectamente como `KEYWORD(ïnt") + IDENTIFIER(.eger")`. El boundary garantiza que `int` solo coincida como palabra completa.

5.2. Implementación en Java

La implementación del analizador léxico se basa en el siguiente diagrama de clases.

5.2.1. Diagrama de clases

En esta sección se presenta el diagrama de clases correspondiente a la implementación del analizador léxico.

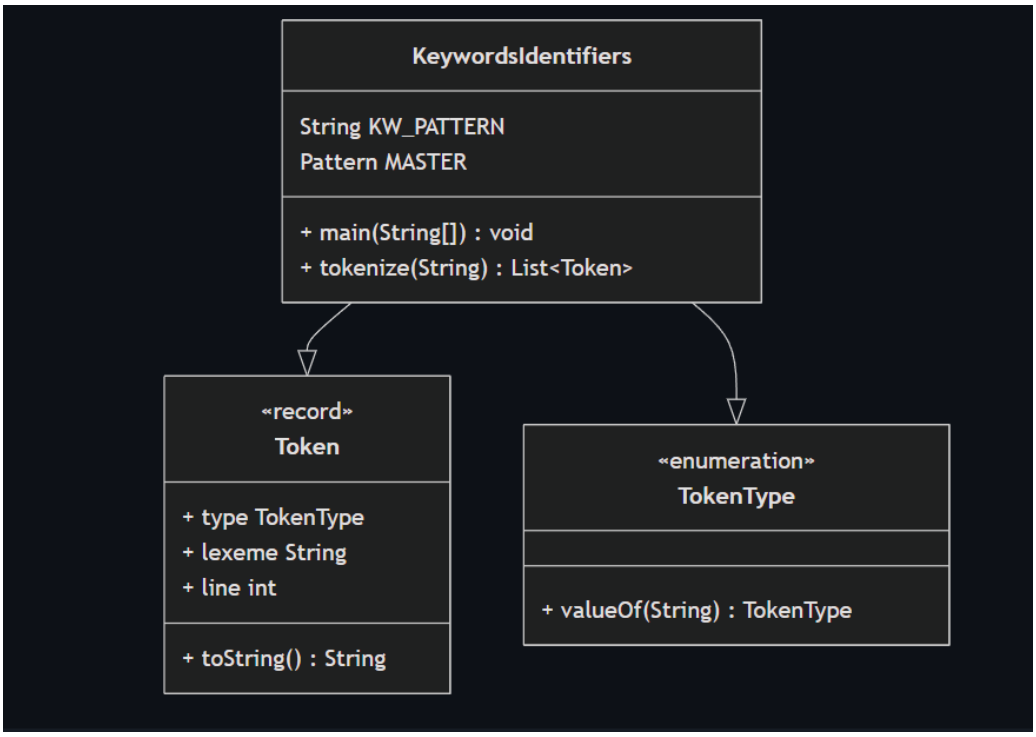


Figura 1: Diagrama de clases de la implementación del analizador léxico.

6. Caso 1 — Básico: Keywords e Identifiers

6.1. Objetivo

Dado un fragmento de código Java, identificar y clasificar cada lexema como `KEYWORD` o `IDENTIFIER`, usando expresiones regulares con la API `Pattern/Matcher` de Java.

6.2. Salida Esperada

En la siguiente figura se muestra la salida obtenida al ejecutar el analizador léxico sobre el fragmento de código Java utilizado en la práctica.

KeywordsIdentifiers.java ¡Corre ▶

```
'\n').count();
            } else if (m.group("KEYWORD") != null) {
                tokens.add(new Token(TokenType.KEYWORD, m.group(), line));
            } else if (m.group("IDENTIFIER") != null) {
                tokens.add(new Token(TokenType.IDENTIFIER, m.group(), line));
            }
            pos = m.end();
        }
        return tokens;
    }

    public static void main(String[] args) {
        String source = """
            public class MiClase {
                private int contador;
                public void incrementar() {
                    contador++;
                }
            }
            """;
        tokenize(source).forEach(System.out::println);
    }
}
```

Producción

```
[L1  ] KEYWORD        ? "public"
[L1  ] KEYWORD        ? "class"
[L1  ] IDENTIFIER     ? "MiClase"
[L1  ] UNKNOWN        ? "{"
[L2  ] KEYWORD        ? "private"
[L2  ] KEYWORD        ? "int"
[L2  ] IDENTIFIER     ? "contador"
[L2  ] UNKNOWN        ? ";"
[L3  ] KEYWORD        ? "public"
[L3  ] KEYWORD        ? "void"
[L3  ] IDENTIFIER     ? "incrementar"
[L3  ] UNKNOWN        ? "()"
[L3  ] UNKNOWN        ? "{"
[L4  ] IDENTIFIER     ? "contador"
[L4  ] UNKNOWN        ? "++;"
[L5  ] UNKNOWN        ? "}"
[L6  ] UNKNOWN        ? "}"
```

Figura 2: Salida esperada del analizador léxico.

6.3. Código modificado

En la siguiente figura se muestra la modificación realizada en la variable `source`, agregando los lexemas `integer`, `forLoop` y `returnValue`.



Figura 3: Modificación de la variable `source`.

6.4. Preguntas de Discusión

1. **¿Qué pasa si eliminas los `\b` del patrón KEYWORD?**

   Si elimino los `\b`, una palabra reservada podría coincidir dentro de un identificador. Por ejemplo, `int` podría coincidir dentro de `integer`, haciendo que el lexer lo clasifique incorrectamente como `KEYWORD(int")` y `IDENTIFIER(.egger")`.

2. **¿Por qué el patrón KEYWORD debe evaluarse antes que IDENTIFIER?**

   El patrón `KEYWORD` debe evaluarse antes que `IDENTIFIER` porque las palabras reservadas también cumplen con la estructura de un identificador. Si se evaluara primero `IDENTIFIER`, palabras como `while`, `int` o `return` podrían ser clasificadas como identificadores.

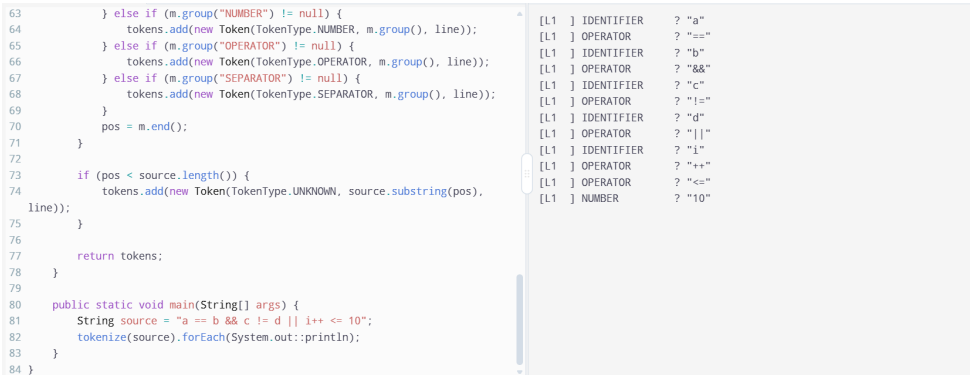
3. **¿Podrías construir una sola ER que reconozca ambos tokens a la vez?**

   Sí, podría construir una sola expresión regular que reconociera tanto palabras reservadas como identificadores, pero necesitaría utilizar grupos de captura para diferenciar qué tipo de token se encontró. Sin embargo, considero más claro utilizar patrones separados para `KEYWORD` e `IDENTIFIER`, ya que permite clasificarlos directamente.

7. Caso 2 — Básico+: Operadores y Separadores

7.1. Objetivo

Reconocer operadores simples y compuestos junto a separadores. Comprender y resolver el **problema del prefijo** con alternancia ordenada. article [utf8]utf8 [spanish]babel graphicx enumitem



```
63              } else if (m.group("NUMBER") != null) {
64                  tokens.add(new Token(TokenType.NUMBER, m.group(), line));
65              } else if (m.group("OPERATOR") != null) {
66                  tokens.add(new Token(TokenType.OPERATOR, m.group(), line));
67              } else if (m.group("SEPARATOR") != null) {
68                  tokens.add(new Token(TokenType.SEPARATOR, m.group(), line));
69              }
70              pos = m.end();
71          }
72
73          if (pos < source.length()) {
74              tokens.add(new Token(TokenType.UNKNOWN, source.substring(pos),
    line));
75          }
76
77          return tokens;
78      }
79
80      public static void main(String[] args) {
81          String source = "a == b && c != d || i++ <= 10";
82          tokenize(source).forEach(System.out::println);
83      }
84  }
```

```
[L1  ] IDENTIFIER     ? "a"
[L1  ] OPERATOR       ? "=="
[L1  ] IDENTIFIER     ? "b"
[L1  ] OPERATOR       ? "&&"
[L1  ] IDENTIFIER     ? "c"
[L1  ] OPERATOR       ? "!="
[L1  ] IDENTIFIER     ? "d"
[L1  ] OPERATOR       ? "||"
[L1  ] IDENTIFIER     ? "i"
[L1  ] OPERATOR       ? "++"
[L1  ] OPERATOR       ? "<="
[L1  ] NUMBER         ? "10"
```

Figura 4: Tokenización del Caso 2

Caso 2

Preguntas de Discusión

¿Cómo manejarías el operador ternario `?:` en tu analizador?

- **En el Analizador Léxico (Lexer):** Se deben tokenizar los símbolos `?` y `:` por separado como tokens independientes de tipo `OPERATOR`. No se deben capturar como un único token `?:` porque el operador ternario es **discontinuo** (hay expresiones intermedias entre ambos caracteres, por ejemplo: `condicion ? valor1 : valor2`).
- **En el Analizador Sintáctico (Parser):** El parser es el encargado de validar la estructura gramatical completa (`ExpresionCondicional` → `ExpresionBooleana '?' Expresion1 ':' Expresion2`), asegurando que cada `?` tenga su correspondiente `:` emparejado.

El operador $>$ (shift sin signo) debe reconocerse antes que . ¿Por qué?

- Por la regla de la **coincidencia más larga (*Longest Match Rule* o *Maximal Munch*)** utilizada en los analizadores léxicos.

- Las expresiones regulares evalúan de izquierda a derecha. Si estuviera antes que $>$, el analizador consumiría únicamente los primeros dos caracteres y los tokenizaría como un desplazamiento a la derecha convencional, dejando el tercer `>` suelto o causando un error sintáctico. Al colocar $>$ primero, el analizador prioriza el token más largo y específico de 3 caracteres.

¿Qué diferencia hay entre un separador y un operador desde la perspectiva de la gramática?

- **Operador:** Representa un cálculo o manipulación de datos sobre uno o más operandos (e.g., `+`, `==`, `&&`). Tienen propiedades de **precedencia**, **asociatividad** y retornan un valor al evaluarse dentro de una expresión.
- **Separador (o Delimitador):** Define la estructura, agrupación y sintaxis del código sin realizar ninguna operación sobre los datos (e.g., `{`, `}`, `;`, `,`, `()`. No tienen precedencia ni devuelven un valor; sirven para delimitar bloques de código, separar parámetros o finalizar sentencias.

article [utf8]utf8 [spanish]babel

Caso 3 — Intermedio: Literales Numéricos y de Cadena

Objetivo

Construir patrones que reconozcan la variedad de literales de Java: enteros decimales, hexadecimales, octales, binarios, flotantes con notación científica y literales de cadena con secuencias de escape.

article [utf8]utf8 [spanish]babel graphicx enumitem

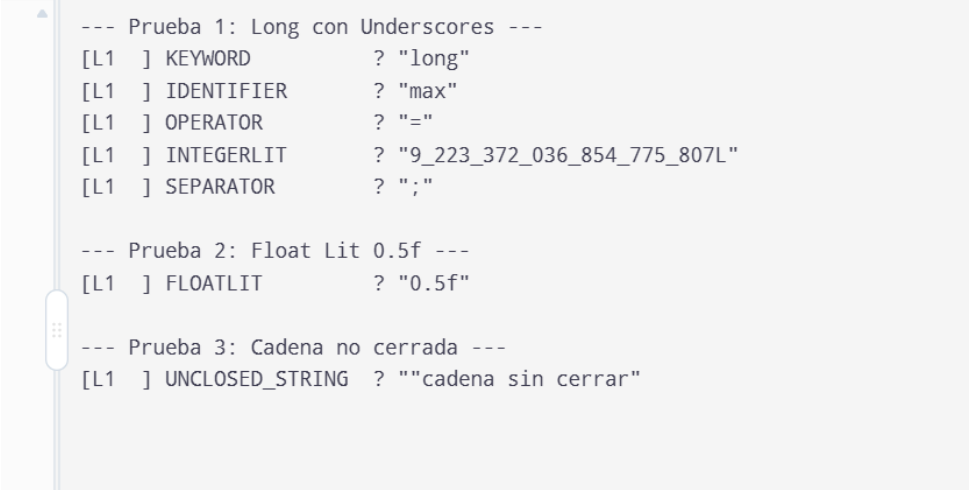


Figura 5: Imagen actividad caso 3

Preguntas de Discusión

¿Por qué `08` NO es un octal válido en Java? ¿Cómo ajustarías la ER?

- **Razón:** En el sistema numérico octal solo se permiten dígitos del **0 al 7**. Al iniciar un literal numérico con el prefijo `0`, Java lo interpreta como una representación en base 8, por lo que el dígito `8` no es un carácter válido dentro de esta base.
- **Ajuste de la Expresión Regular:** Definiendo el patrón octal como `0[0-7]+`, el literal `08` no coincidirá con la regla de octales. El analizador léxico procesará el `0` como un número entero independiente y el `8` como otro token separado o un error léxico.

¿Cómo extenderías el patrón `STRING` para soportar Text Blocks de Java 15+ (`"..."`)?

- Se debe añadir una rama en la Expresión Regular con el modificador `(?s)` (*dotall mode*) para permitir que el punto coincida con saltos de línea (`\n`):

```
"""(?s).*?"""
```

- El uso del cuantificador no codicioso (*lazy*) `*?` asegura que la captura finalice correctamente en el primer conjunto de tres comillas dobles de cierre que encuentre.

¿Tiene sentido que el analizador léxico verifique si un número está en rango (e.g., `int >2^31`)?

- **No.** La función principal del analizador léxico (Lexer) es la **verificación sintáctica regular** (validar el formato de la secuencia de caracteres), no el análisis de valores ni la evaluación semántica.
- Validar si un valor numérico excede los límites permitidos del tipo de dato $(2^{31} - 1)$ le corresponde a la fase de **Análisis Semántico** o a la construcción del árbol de sintaxis abstracta (AST) en el compilador.

article [utf8]utf8 [spanish]babel

05 · Caso 4 — Intermedio: Comentarios y Errores Léxicos

Objetivo

Reconocer y descartar comentarios (de línea y de bloque), manejar whitespace con conteo de líneas y reportar errores léxicos con línea y columna cuando se encuentra un carácter no reconocido.

8. Implementación del Código Completo

A continuación se presenta el código completo del analizador léxico desarrollado durante la práctica. Debido a la extensión del programa, se dividió en cinco capturas para facilitar su visualización.

```
import java.util.regex.*;
import java.util.*;
import java.util.stream.*;

public class JavaLexer {

    enum TokenType {
        JAVADOC, BLOCKCOMMENT, LINECOMMENT,
        KEYWORD, IDENTIFIER,
        OPERATOR, SEPARATOR,
        FLOATLIT, HEXADECIMALLIT, BINARYLIT, OCTALLIT, INTEGERLIT
    ,
        STRINGLIT, CHARLIT,
        WHITESPACE, UNKNOWN
    }

    record Token(TokenType type, String lexeme, int line, int
    column) {
        public String toString() {
            return String.format("[L%-3d C%-3d] %-14s -> \"%s\"",
                    line, column, type, lexeme);
        }
        public String toCsv() {
            String lex = lexeme.replace("\"", "\"\"");
            return String.format("%d,%d,%s,\"%s\"", line, column,
    type, lex);
        }
    }

    public static class LexicalError extends RuntimeException {
        private final int line, column;
        private final char illegal;

        public LexicalError(char c, int line, int col) {
            super(String.format(
                    "Error lexico [linea %d, col %d]: caracter
    ilegal '%c' (U+%04X)",
                    line, col, c, (int) c));
            this.line = line; this.column = col; this.illegal = c
    ;
        }
        public int getLine()   { return line; }
        public int getColumn() { return column; }
        public char getIllegal(){ return illegal; }
    }

    static final String KW_PATTERN =
            "\\b(abstract|assert|boolean|break|byte|case|catch|
    char|" +
```

Figura 6: Código completo del analizador léxico – Parte 1.

```
            "class|const|continue|default|do|double|else|enum|
extends|" +
            "final|finally|float|for|if|implements|import|
instanceof|" +
            "int|interface|long|new|package|private|protected|
public|" +
            "return|short|static|super|switch|synchronized|this|
throw|" +
            "throws|try|void|volatile|while|true|false|null)\\b";

    static final Pattern MASTER = Pattern.compile(
            "(?<JAVADOC>/\\*\\*[\\s\\S]*?\\*/)|"       +
            "(?<BLOCKCOMMENT>/\\*[\\s\\S]*?\\*/)|"     +
            "(?<LINECOMMENT>//[^\\n]*)|"               +
            "(?<STRINGLIT>\"([^\"\\\\]|\\\\.)*\")|"    +
            "(?<CHARLIT>'([^'\\\\]|\\\\(u[0-9a-fA-F]{4}|.))')|" +
            "(?<FLOATLIT>[0-9][0-9_]*\\.[0-9][0-9_]*([eE
][+-]?[0-9]+)?[fFdD]?)|" +
            "(?<HEXADECIMALLIT>0[xX][0-9a-fA-F][0-9a-fA-F_]*[lL
]?)|" +
            "(?<BINARYLIT>0[bB][01][01_]*[lL]?)|"      +
            "(?<OCTALLIT>0[0-7]+[lL]?)|"                +
            "(?<INTEGERLIT>0|[1-9][0-9_]*[lL]?)|"      +
            "(?<KEYWORD>" + KW_PATTERN + ")|"          +
            "(?<IDENTIFIER>[a-zA-Z_$][a-zA-Z0-9_$]*)|" +
            "(?<OPERATOR>" +
            "==|!=|<=|>=|&&|\\|\\||<<=|>>>=|>>=|<<|>>>|>>|" +
            "\\+\\+|--|\\+=|-=|\\*=|/=|%=|&=|\\|=|\\^=|" +
            "[+\\-*/%<>=!&|^~?:]" +
            ")|" +
            "(?<SEPARATOR>[(){}\\[\\];,.])|"           +
            "(?<WHITESPACE>[ \\t\\r\\n]+)",
            Pattern.MULTILINE
    );

    public static List<Token> tokenize(String source, List<
LexicalError> errors) {
        List<Token> tokens = new ArrayList<>();
        Matcher m = MASTER.matcher(source);
        int line = 1, pos = 0;

        while (m.find()) {
            if (m.start() > pos) {
                reportErrors(source, pos, m.start(), line, errors
);
                line += countNewlines(source.substring(pos, m.
start()));
            }

            if (m.group("WHITESPACE")    != null ||
                m.group("JAVADOC")       != null ||
```

Figura 7: Código completo del analizador léxico – Parte 2.

```
                m.group("BLOCKCOMMENT") != null) {
                line += countNewlines(m.group());
            }

            if (m.group("KEYWORD") != null) {
                tokens.add(mk(TokenType.KEYWORD, m, source, line)
);
            } else if (m.group("IDENTIFIER") != null) {
                tokens.add(mk(TokenType.IDENTIFIER, m, source,
line));
            } else if (m.group("OPERATOR") != null) {
                tokens.add(mk(TokenType.OPERATOR, m, source, line
));
            } else if (m.group("SEPARATOR") != null) {
                tokens.add(mk(TokenType.SEPARATOR, m, source,
line));
            } else if (m.group("FLOATLIT") != null) {
                tokens.add(mk(TokenType.FLOATLIT, m, source, line
));
            } else if (m.group("HEXADECIMALLIT") != null) {
                tokens.add(mk(TokenType.HEXADECIMALLIT, m, source
, line));
            } else if (m.group("BINARYLIT") != null) {
                tokens.add(mk(TokenType.BINARYLIT, m, source,
line));
            } else if (m.group("OCTALLIT") != null) {
                tokens.add(mk(TokenType.OCTALLIT, m, source, line
));
            } else if (m.group("INTEGERLIT") != null) {
                tokens.add(mk(TokenType.INTEGERLIT, m, source,
line));
            } else if (m.group("STRINGLIT") != null) {
                tokens.add(mk(TokenType.STRINGLIT, m, source,
line));
            } else if (m.group("CHARLIT") != null) {
                tokens.add(mk(TokenType.CHARLIT, m, source, line)
);
            }

            pos = m.end();
        }

        if (pos < source.length()) {
            reportErrors(source, pos, source.length(), line,
errors);
        }

        return tokens;
    }
```

Figura 8: Código completo del analizador léxico – Parte 3.

```
    private static Token mk(TokenType t, Matcher m, String s,
int line) {
        return new Token(t, m.group(), line, colOf(src, m.st
));
    }

    private static long countNewlines(String s) {
        return s.chars().filter(c -> c == '\n').count();
    }

    private static int colOf(String source, int pos) {
        int nl = source.lastIndexOf('\n', pos - 1);
        return pos - nl;
    }

    private static void reportErrors(String source, int star
int end,
                                     int line, List<LexicalE
    errors) {
        int col = colOf(source, start);
        for (int i = start; i < end; i++) {
            char c = source.charAt(i);
            LexicalError err = new LexicalError(c, line, col
    - start));
            errors.add(err);
            System.err.println("ALERT: " + err.getMessage()
        }
    }

    public static Map<TokenType, Long> frequency(List<Token>
tokens) {
        return tokens.stream().collect(Collectors.groupingBy
                Token::type, TreeMap::new, Collectors.counti
);
    }

    public static String toCsv(List<Token> tokens) {
        StringBuilder sb = new StringBuilder("linea,columna,
lexema\n");
        for (Token t : tokens) sb.append(t.toCsv()).append("
        return sb.toString();
    }
}
```

Figura 9: Código completo del analizador léxico – Parte 4.

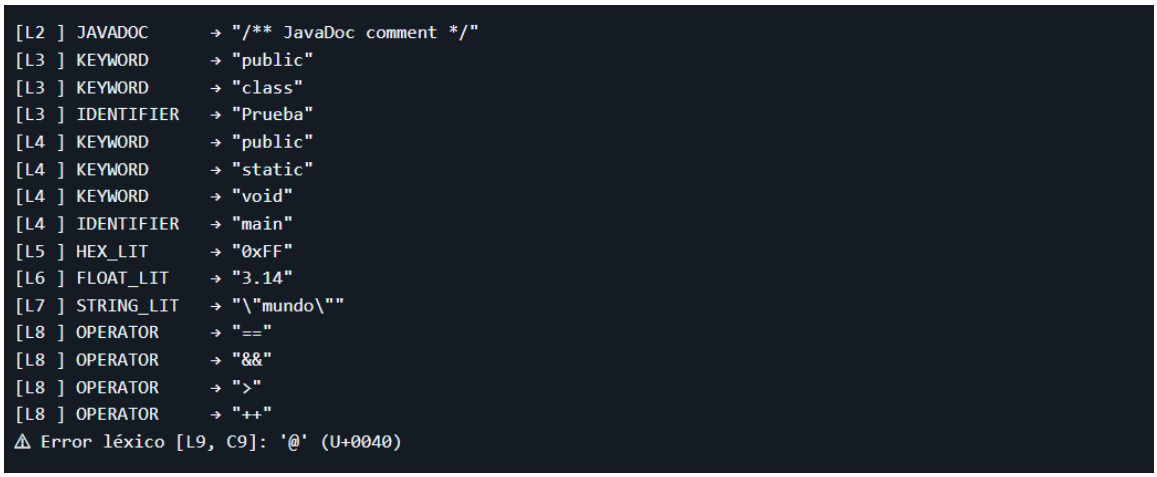


Figura 10: Salida esperada

Cuadro 1: Resumen estadístico del análisis léxico

Tipo de Token (TokenType)	Cantidad Identificada
FLOATLIT	1
HEXADECIMALLIT	1
IDENTIFIER	6
INTEGERLIT	1
KEYWORD	7
OPERATOR	5
SEPARATOR	9
STRINGLIT	1

8.1. Tokens identificados por tipo

A continuación se muestran los tokens correspondientes a cada una de las categorías reconocidas por el analizador léxico:

- **FLOATLIT:** `3.14`
- **HEXADECIMALLIT:** `0xFF`
- **IDENTIFIER:** `Prueba`, `main`, `args`, `x`, `pi`, `s`
- **INTEGERLIT:** `255`
- **KEYWORD:** `public`, `class`, `static`, `void`, `int`, `double`, `if`
- **OPERATOR:** `=`, `==`, `&&`, `>`, `++`
- **SEPARATOR:** `{`, `}`, `(`, `)`, `[`, `]`, `;`, `,`, `.`
- **STRINGLIT:** `"mundo"`

9. Preguntas de Discusión

9.1. ¿Cómo manejaría tu lexer un comentario de bloque no cerrado al final del archivo?

Si el lexer encuentra el inicio de un comentario de bloque mediante los caracteres `/*`, pero no encuentra los caracteres de cierre `*/` antes de llegar al final del archivo, debe identificar esta situación como un error léxico.

9.2. ¿Deberían los JavaDoc comments producir un token diferente a los block comments?

Depende del propósito del analizador léxico. Si el objetivo principal es identificar los elementos necesarios para el análisis sintáctico, tanto los comentarios JavaDoc como los comentarios de bloque pueden ser ignorados, ya que normalmente no forman parte de la estructura sintáctica del programa.

9.3. ¿Cuál es la diferencia entre recuperación de errores en el lexer vs. en el parser?

La recuperación de errores en el lexer y en el parser ocurre en diferentes etapas del proceso de compilación.

El **lexer** trabaja directamente con los caracteres del código fuente y se encarga de convertirlos en tokens.

Por otro lado, el **parser** trabaja con los tokens generados por el lexer y verifica que estos cumplan con las reglas gramaticales del lenguaje. Por lo tanto, la principal diferencia es que el lexer recupera errores relacionados con la formación de tokens, mientras que el parser recupera errores relacionados con la estructura sintáctica del programa.